

File permissions in Linux

Project description

The research team at my organization needs to update the file permissions for certain files and directories within the `projects` directory. The permissions do not currently reflect the level of authorization that should be given. Checking and updating these permissions will help keep their system secure. To complete this task, I performed the following tasks:

Check file and directory details

The following code demonstrates how I used Linux commands to determine the existing permissions set for a specific directory in the file system:

```
researcher2@7b647816afb2:~$ cd projects
researcher2@7b647816afb2:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 17:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 17:56 ..
-rw--w---- 1 researcher2 research_team  46 Mar  5 17:16 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Mar  5 17:16 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Mar  5 17:16 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Mar  5 17:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 17:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 17:16 project_t.txt
```

The first line of the screenshot shows the command I executed, while the following lines show the resulting output. The command displays all items within the `projects` directory. I used the `ls` command with the `-la` option to generate a detailed list of the directory contents, including hidden files. The output reveals one directory called `drafts`, one hidden file named `.project_x.txt`, and five additional project files. The 10-character string in the first column indicates the permission settings assigned to each file or directory.

Describing the permissions string

The 10-character permissions string can be broken down to understand who has access to a file and what type of access they have. Each position in the string has a specific meaning:

- **1st character:** This is either a `d` or a hyphen (`-`) and shows the file type. A `d` means it is a directory, while a hyphen (`-`) means it is a regular file.
- **2nd–4th characters:** These represent the user's permissions — read (`r`), write (`w`), and execute (`x`). If any of these positions contain a hyphen (`-`), it means that specific permission is not granted to the user.

- **5th–7th** characters: These show the group’s permissions — read (r), write (w), and execute (x). Again, a hyphen (-) indicates the group does not have that permission.
- **8th–10th** characters: These indicate the permissions for others, meaning all remaining users on the system who are not the owner or part of the group. As before, a hyphen (-) means the permission is not granted.

For example, the permissions for `project_k.txt` are `-rw-rw-rw-`. The first character is a hyphen, so this confirms it is a regular file. The letters r in the second, fifth, and eighth positions show that the user, group, and others all have read access. The w in the third, sixth and ninth positions means all of the owner types can write to the file. Since there are no x characters, no one has execute permission for this file.

Changing file permissions

The organization decided that users classified as “other” should not have write permissions for any files. To address this requirement, I reviewed the file permissions I had previously displayed and identified that `project_k.txt` needed to have write access removed for others.

The following code shows how I used Linux commands to make this change:.

```
researcher2@7b647816afb2:~/projects$ chmod o-w project_k.txt
researcher2@7b647816afb2:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 17:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 17:56 ..
-rw--w---- 1 researcher2 research_team  46 Mar  5 17:16 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Mar  5 17:16 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 17:16 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Mar  5 17:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 17:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 17:16 project_t.txt
researcher2@7b647816afb2:~/projects$
```

The first two lines of the screenshot show the commands I executed, while the remaining lines display the output from the second command. The `chmod` command is used to modify permissions for files and directories. The first argument defines which permissions should be adjusted, and the second argument identifies the specific file or directory. In this case, I removed write permission for others from the `project_k.txt` file. I then ran `ls -la` to verify that the changes were successfully applied.

Changing file permissions on a hidden file

The research team at my organization recently archived the file `.project_x.txt`. They decided that no one should have write permissions for this file, while the user and group should still retain read access.

The following code shows how I used Linux commands to update the permissions accordingly:

```
researcher2@cbd6c19ce834:~/projects$ chmod u-w,g+r-w .project_x.txt
researcher2@cbd6c19ce834:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 18:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 19:04 ..
-r--r----- 1 researcher2 research_team  46 Mar  5 18:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Mar  5 18:27 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Mar  5 18:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_t.txt
researcher2@cbd6c19ce834:~/projects$
```

The first two lines of the screenshot show the commands I executed, and the remaining lines present the output of the second command. I recognized that `.project_x.txt` is a hidden file because its name begins with a period (.). In this case, I removed write permissions from both the user and the group, and ensured the group had read access. I used `u-w` to remove write permission from the user, `g+r-w` to grant read permission to the group and to remove write permission.

Changing directory permissions

My organization decided that only the user `researcher2` should be able to access the `drafts` directory and everything inside it. Therefore, no other users should have execute permissions for this directory.

The following code shows how I used Linux commands to modify the permissions accordingly:

```
researcher2@cbd6c19ce834:~/projects$ chmod g-x drafts
researcher2@cbd6c19ce834:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 18:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Mar  5 19:04 ..
-r--r----- 1 researcher2 research_team  46 Mar  5 18:27 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Mar  5 18:27 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Mar  5 18:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Mar  5 18:27 project_t.txt
researcher2@cbd6c19ce834:~/projects$
```

The first two lines of the screenshot show the commands I ran, and the remaining lines display the output of the second command. Earlier, I identified that the group had execute permissions, so I used the `chmod` command to remove them. The user `researcher2` already had execute permissions, so no additional changes were needed for that user.

Summary

I adjusted several permissions to ensure they aligned with the level of access my organization required for the files and directories within the projects folder. I began by running `ls -la` to review the existing permissions, which guided my decisions in the next steps. After that, I used the `chmod` command multiple times to update the permissions for specific files and directories.